

MedNuskha

External setup guide — everything the team does outside Claude Code

Team Runtime Terrors · Alibaba Cloud AI Hackathon 2026 · 22 August 2026

Claude Code writes the code. It cannot click through Meta's dashboard, create your accounts, or accept anyone's terms of service. **This document is that half of the job.**

Work top to bottom. Section 2 (WhatsApp templates) has a **multi-hour approval wait** attached to it, so start it first and do everything else while the queue moves.

Order of play

When	What	Blocks
Right now	Submit the 4 WhatsApp message templates (§2.6)	Phase 1 — nothing sends without these
Today	Meta app, permanent token, whitelist phones (§2.1–2.5)	Phase 1
Today	Supabase project + storage bucket (§1)	Every phase — there is no database yet
Today	DashScope API key (§3)	Phase 3 (agent) and Phase 7 (OCR)
Before Phase 1 test	ngrok account + webhook config (§4)	Receiving replies
Before Phase 5	Google Cloud Text-to-Speech (§5)	Voice notes only
Day 4	Alibaba ECS + Vercel deploy (§6)	The live demo

Where every value ends up

All of it goes into one file: **.env** in the project root. That file is gitignored and will never reach GitHub. Never paste a real secret into chat, a commit, or a screenshot.

Variable in .env	Comes from	Section
DATABASE_URL	Supabase → Connect → Session pooler	1
SUPABASE_URL	Supabase → Settings → API	1
SUPABASE_ANON_KEY	Supabase → Settings → API (publishable key)	1
SUPABASE_STORAGE_BUCKET	Already set to <i>voice-notes</i>	1
WHATSAPP_PHONE_NUMBER_ID	Meta → WhatsApp → API Setup	2.3
WHATSAPP_WABA_ID	Meta → WhatsApp → API Setup	2.3
WHATSAPP_TOKEN	Business Settings → System User token	2.5
WHATSAPP_VERIFY_TOKEN	You invent it (already generated)	2.5
WHATSAPP_API_VERSION	Already set to v23.0	—
DASHSCOPE_API_KEY	Alibaba Model Studio	3
GOOGLE_APPLICATION_CREDENTIALS	Path to a Google service-account .json	5

Variable in .env	Comes from	Section
NEXT_PUBLIC_API_BASE	Your backend URL (localhost until Phase 8)	6

1 Supabase — the database

Free tier. This is where all eleven tables and every voice note live.

1.1 Create the project

1. Go to **supabase.com** and sign in with GitHub.
2. **New project**. Name it **MedNuskha**.
3. Set a database password. **Use letters and numbers only** — see the warning below.
4. Region: closest to Pakistan you can get. Any region works; it only costs a little latency.
5. Wait about two minutes while it provisions.

DO NOT SKIP

The database password is shown **once**, at creation, and it is part of your connection string. Save it somewhere safe immediately. If you lose it you must reset it in Settings → Database.

Avoid symbols in it. A password containing @ # \$ / : ? breaks the connection string unless it is URL-encoded, and that failure looks like a wrong password.

1.2 Get DATABASE_URL — there is a real choice here

Click **Connect** at the top of the project, then the **Direct connection string** tab. You will be offered three. They are not interchangeable:

Option	Port	Use it?
Direct connection	5432	No. IPv6-only on new projects — on most home and office networks it simply will not connect.
Session pooler	5432	Yes — this one. IPv4, and the right mode for a long-running FastAPI process that holds a connection pool.
Transaction pooler	6543	Works (the code already disables prepared statements for it), but session mode is the better fit. Use only as a fallback.

It looks like this:

```
postgresql://postgres.jhustfytfatkr1qudgcz:[YOUR-PASSWORD]@aws-0-ap-northeast-2.pooler.supabase.com:5432/postgres
```

Replace [YOUR-PASSWORD] with your real password — delete the square brackets too. Then paste the whole line into .env after DATABASE_URL=.

TIP

Paste it exactly as Supabase gives it. The code rewrites postgresql:// to postgresql+psycopg:// for you — you do not need to edit the scheme.

1.3 Get the two API values

Project Settings → API:

- **Project URL** → SUPABASE_URL (looks like https://xxxx.supabase.co)
- **Publishable key**, starting sb_publishable_ → SUPABASE_ANON_KEY. Supabase renamed the “anon key” to “publishable key”; they are the same thing.

1.4 Create the storage bucket

Left sidebar → **Storage** → **New bucket** → name it exactly **voice-notes** → leave it **Private** → Create.

HEADS UP

While you are on the API page, also copy the **secret key** (sb_secret_...) into your private notes. **Do not put it in .env yet.**

A private bucket refuses writes from the publishable key, so Phase 5 will not be able to upload voice notes without it. Grabbing it now saves a trip back to the dashboard on Day 3.

1.5 Prove it worked

From the project folder, run `.\dev.ps1` and open `http://localhost:3000`.

WHEN IT WORKS

The card on that page flips to **database: connected**.

That is not cosmetic — it means all eleven tables were just created in Supabase automatically on first boot. Confirm them under **Table Editor**. You never run a migration by hand.

Do not skip this check. Everything after this point assumes the database answers.

2 Meta WhatsApp Cloud API

The biggest piece of external work, and the one with a queue attached. Free: Meta's Cloud API direct, with a free test phone number. No SIM card, no BSP, no payment method.

DO NOT SKIP

Do §2.6 (submit the templates) first, before anything else in this section. Approval takes hours and sometimes a day, and no dose reminder can be sent until dose_reminder is approved. Everything else here takes about 20 minutes and can happen while you wait.

Two different Meta websites are involved, which is the main source of confusion:

- **developers.facebook.com** — the app, the test number, the webhook
- **business.facebook.com** — the permanent token and the message templates

2.1 Create the Meta app

1. Go to **developers.facebook.com** and log in with a Facebook account.
2. **My Apps** → **Create App**.
3. Use case: **Other** → Next. App type: **Business** → Next.
4. Name it **MedNuskha**, add your email, and pick or create a Business Account.

2.2 Add the WhatsApp product

In the app dashboard, find **WhatsApp** under “Add products” and click **Set up**. Meta issues you a free test phone number automatically — that is the number your patients will receive messages from during the hackathon.

2.3 Copy two IDs

You are now on **WhatsApp** → **API Setup**. Copy these into .env:

```
WHATSAPP_PHONE_NUMBER_ID= <-- "Phone number ID"
WHATSAPP_WABA_ID= <-- "WhatsApp Business Account ID"
```

HEADS UP

The same page offers a **temporary access token**. **Ignore it**. It expires in 24 hours, which means it will die at the worst possible moment. The permanent one is \$2.5.

2.4 Whitelist your test phones

Still on API Setup: the **To** dropdown → **Manage phone number list** → add up to **five** numbers. Each receives a code on WhatsApp that you type back in to confirm.

- Add **your number and two teammates** — and keep two slots spare.
- Numbers go in international format, e.g. +92 300 1234567.

DO NOT SKIP

The test number can only message these five numbers. Anyone not on the list receives absolutely nothing, and the API still returns success. This is the single most common cause of “why is nothing arriving”.

Keep a spare whitelisted number in reserve — heavy testing can get one rate-limited, and you do not want that to be your demo phone.

2.5 The permanent token

This happens on **business.facebook.com**, not the developers site.

1. **business.facebook.com** → **Business Settings** (the gear icon).
2. Sidebar → **Users** → **System Users** → **Add**.
3. Name it **mednuskha-bot**, role **Admin** → Create.
4. **Assign Assets** → **Apps** → select MedNuskha → toggle **Full control** → Save.
5. **Assign Assets** again → **WhatsApp Accounts** → select your WABA → toggle **Full control** → Save.
6. **Generate New Token** → pick the MedNuskha app.
7. Set **Token expiration: Never**.
8. Tick exactly these two permissions:
 - whatsapp_business_messaging
 - whatsapp_business_management

Generate, then copy the token into .env as `WHATSAPP_TOKEN=`.

DO NOT SKIP

The token is displayed once. If you close that dialog without copying it, you generate a new one and start over. Paste it into .env before you click anything else.

Your verify token

This one you invent — Meta only has to echo it back. Already generated for you; paste this line into .env as-is:

```
WHATSAPP_VERIFY_TOKEN=mednuskha_9-lZAnI7pUlb8VagUFqG62I9uZu0YyZp
```

2.6 Submit the four message templates — DO THIS FIRST

business.facebook.com → **WhatsApp Manager** → **Message Templates** → **Create Template**. Repeat four times.

Every one of these is **Category: UTILITY**. Not Marketing — Marketing templates get throttled, cost money, and can be blocked by the user.

#	Template name	Body text	Buttons
1	dose_reminder	{{1}} ji, {{2}} baj gaye — {{3}} lene ka waqt hai. {{4}}	2 Quick Reply
2	dose_followup	{{1}} ji, sirf yaad dila rahe hain — {{2}} abhi baaki hai.	2 Quick Reply
3	caretaker_alert	{{1}} ne aaj {{2}} ki {{3}} confirm nahi ki. Do baar yaad dilaya gaya hai.	none
4	patient_optin	Assalam-o-alaikum {{1}} ji. {{2}} ne aap ke liye dawai ki yaad-dahani set ki hai. Shuru karne ke liye “HAAN” likhein.	none

The two Quick Reply buttons

On templates 1 and 2, add two **Quick Reply** buttons, in this exact order:

- Button 1: **Le li** (add a check-mark symbol after it if you like)
- Button 2: **Abhi nahi**

DO NOT SKIP

Order matters. “Le li” must be the first button. The code identifies which button was tapped by its position, so swapping them makes every “I took it” register as “not yet”.

You only set the button *labels* here. The dose ID that travels in the button payload is attached at send time by the backend — there is nothing for you to configure.

Four things that get templates rejected

Trap	What to do
Sample values are mandatory	Meta will not let you submit until every {{1}} has an example. Use: Abdul / 8 / Panadol 500mg / Khane ke baad lein
Language choice	Pick English for all four. The text is Roman Urdu but written in Latin letters, and reviewers approve it faster than under “Urdu”, which expects Urdu script. Use the same language for all four and tell Claude which you picked — the code sends that exact code to Meta, and a mismatch fails silently.
Symbols in button labels	If the check mark causes a rejection, resubmit as plain Le li and tell Claude you changed it.
Wrong category	UTILITY, every time. A reminder the user opted into is a utility message, and Meta agrees — but only if you tell it so.

WHEN IT WORKS

All four show **Approved** in WhatsApp Manager. Usually 1–4 hours, occasionally a full day.

If they are still pending when you want to test: message the test number from your phone first. That opens a 24-hour service window in which plain text can be sent without a template, which is enough to keep development moving.

3 Alibaba Model Studio (DashScope) — Qwen

Powers the agent's understanding of messy replies (Phase 3), the medicine lookup (Phase 4), and prescription OCR (Phase 7). Covered by the hackathon credits.

1. Go to **dashscope.aliyun.com** (Alibaba Cloud Model Studio) and create an account.
2. Redeem the **hackathon credit code** given to your team.
3. Activate the Model Studio service if prompted.
4. Create an **API key** and copy it into .env as DASHSCOPE_API_KEY=.

HEADS UP

Check which **region** your key belongs to (Singapore and Beijing are separate endpoints with separate keys). If Phase 3 returns authentication errors that look wrong, a region mismatch is the usual reason — tell Claude which region you signed up in.

4 ngrok and the webhook

Do this only once Claude has finished Phase 1. Until the webhook code exists there is nothing for Meta to verify against, and the setup will fail.

Meta has to reach your laptop over HTTPS to deliver replies. ngrok gives you a public HTTPS URL that forwards to your local backend.

4.1 Install ngrok

1. Sign up free at **ngrok.com**.
2. Install it, then run the `ngrok config add-authtoken` command shown on your dashboard.
3. With the backend already running, start the tunnel:

```
ngrok http 8000
```

Copy the `https://` URL it prints.

4.2 Point Meta at it

developers.facebook.com → **your app** → **WhatsApp** → **Configuration** → **Webhook** → **Edit**

- **Callback URL:** `https://<your-ngrok-url>/webhook`
- **Verify token:** the `mednuskha_9-1Z...` string from §2.5
- Click **Verify and save**. The backend must be running for this to pass.
- Then **Manage** → subscribe to the **messages** field.

DO NOT SKIP

The **ngrok URL changes every time you restart ngrok**, and the webhook then stops delivering *silently* — no error, messages just stop arriving.

Every time you restart ngrok, go back into Meta and re-save the new URL. This costs somebody an hour at least once per hackathon. Do not let it be an hour on Day 4.

WHEN IT WORKS

`python scripts/send_test.py 92300XXXXXX` puts a real template on a real phone, and tapping a button writes a row into the database. That is the Phase 1 gate.

5 Google Cloud Text-to-Speech — Urdu voice

Only needed for Phase 5. Skip it until then. Free tier, roughly 1–4 million characters per month, which is far more than a demo consumes.

1. Create a project at **console.cloud.google.com**.
2. Enable the **Cloud Text-to-Speech API**.
3. **IAM & Admin** → **Service Accounts** → **Create**. Give it the **Cloud Text-to-Speech User** role.
4. Create a **JSON key** for that service account and download it.

5. Save the file **outside the repo**, or somewhere gitignored, and point `GOOGLE_APPLICATION_CREDENTIALS` at its full path.

HEADS UP

Google requires a **card on file** to unlock the free tier. Nothing is charged inside the quota, but if putting a card down is a problem for your team, say so early — there is a free offline fallback (Piper TTS) that needs no account at all, at somewhat lower voice quality.

TIP

Before Phase 5, run `faster-whisper` once locally on any Urdu audio clip. That downloads and caches the model, so the first patient voice note is not waiting on a download. Speech recognition runs entirely on your machine — no key, no quota, no bill.

6 Deployment — Day 4

The demo must not run from someone's laptop. Phase 8 moves it to a real server and a real URL.

6.1 Backend on Alibaba Cloud ECS

1. Provision an ECS instance on the hackathon credits: **2 vCPU / 4 GB, Ubuntu 22.04**.
2. Open ports **80** and **443** in the security group.
3. Deploy the backend there — Docker, or copy the files and run it under `systemd`. Use whichever your team is faster at under pressure.
4. Point a subdomain at the instance IP.
5. Get HTTPS working: **Caddy** does it automatically, or `certbot` with `nginx`.
6. Set every environment variable on the server. Never commit them.

WHEN IT WORKS

WeasyPrint (the PDF engine) does not run on Windows without extra system libraries — but on Ubuntu it works out of the box. Moving to the ECS box fixes that problem rather than creating one.

6.2 Dashboard on Vercel

1. From the `frontend/` folder, run `vercel --prod`.
2. In Vercel's project settings, set `NEXT_PUBLIC_API_BASE` to your backend's HTTPS URL.
3. Redeploy so the variable takes effect.

6.3 Repoint WhatsApp and retire ngrok

1. Meta → WhatsApp → Configuration → Webhook: change the callback URL to `https://<your-backend>/webhook` and verify again.
2. Send yourself a reminder from the deployed instance to confirm it works.
3. Stop `ngrok`. It has done its job.

HEADS UP

A Supabase free-tier project **pauses itself after inactivity**. Set a cron on the ECS box to hit the backend every few minutes so the database is awake when the judges are watching.

7 When something breaks

Symptom	Almost always this
Message sends without error, never arrives	The recipient is not on the 5-number whitelist (§2.4). The API returns success either way.
Worked yesterday, dead today	The 24-hour temporary token expired. You need the permanent System User token (§2.5).
Replies stopped arriving, nothing in the logs	ngrok restarted and the URL changed. Re-save it in Meta (§4.2).
Webhook verification fails	The backend is not running, or the verify token does not match .env exactly.
Template will not send	Not approved yet, or the language code in the code does not match the language you submitted it under.
Dashboard says database: not connected	Password still has [YOUR-PASSWORD] in it, a symbol needs URL-encoding, or you used the Direct connection instead of the Session pooler (§1.2).
Database was fine, now times out	Supabase paused the project after inactivity. Open the dashboard to wake it.
The same reply is processed twice	It is not — the duplicate is rejected on purpose by a unique index. That is correct behaviour.

8 Tick-off checklist

Blocking — today

- ☐ All four WhatsApp templates **submitted** (§2.6)
- ☐ All four templates **approved**
- ☐ Meta app created, WhatsApp product added
- ☐ Five test numbers whitelisted and confirmed
- ☐ Permanent System User token in .env
- ☐ Supabase project created, password saved safely
- ☐ DATABASE_URL, SUPABASE_URL, SUPABASE_ANON_KEY in .env
- ☐ Private **voice-notes** bucket created
- ☐ Supabase secret key saved in private notes (needed Phase 5)
- ☐ DashScope API key in .env
- ☐ .\dev.ps1 shows **database: connected**

Before the Phase 1 test

- ☐ ngrok installed and authenticated
- ☐ Webhook URL and verify token saved in Meta
- ☐ Subscribed to the **messages** field
- ☐ A real template message received on a real phone

Later phases

- ☐ Google Cloud TTS service-account JSON downloaded (Phase 5)
- ☐ faster-whisper warmed up locally on an Urdu clip (Phase 5)
- ☐ Two or three real prescriptions photographed (Phase 7)
- ☐ Alibaba ECS instance running with HTTPS (Phase 8)

- [] Dashboard deployed to Vercel (Phase 8)
- [] Webhook repointed at the deployed backend (Phase 8)
- [] Keep-alive cron hitting Supabase (Phase 8)

Two rules that hold for all of it. Never commit a secret — .env is gitignored, and only variable *names* ever belong in .env.example. And when anything here fails, tell Claude the exact error text rather than a summary: the difference between a wrong password, an IPv6 problem and a firewall block is visible in the message, and each has a different fix.